

Checking Failure-Path Telemetry in the Data a Project Already Emits

Halley Young
Preprint, October 2026

Abstract—When a request fails in production, responders need the emitted telemetry to say which request failed and why. We present **TELEMETRY-CONTRACTS**, a pure-Python command-line tool that checks this property in the logs, traces and metrics a project already has, with no contract written in advance and no instrumentation convention to adopt. It defines six gap classes with written definitions and checks each one as a deterministic function of the events. On a 112-item gold set the tool reaches F1 0.953. Two zero-shot LLM judges given the same class definitions reach 0.991 (Claude Haiku 4.5) and 0.952 (GPT-4.1-mini), and neither difference is statistically significant (exact McNemar $p = 0.125$ and $p = 1.00$). The tool reaches this accuracy offline and deterministically at no per-event cost, and it outperforms two rule baselines with $p < 0.001$. We ran it over 58 pinned public repositories (116,420 events, 2,269 findings) in 8.5 minutes on a laptop. In a stratified sample of 60 findings validated by LLM-assisted inspection, the two failure-path checks matched their definitions in 25 of 30 cases. The tool, gold set, corpus manifest, prompts, raw model outputs, validation labels and a blinded annotation packet for independent raters are released.

I. INTRODUCTION

Logging research has long studied whether code logs enough to diagnose failures. It has measured logging practice [1], [2], [3], added diagnostic variables to existing log statements [4], added log statements on error paths [5], and predicted where developers should log [6], [7], [8]. These approaches act on source code. They answer the question “should this code location log?”

This paper asks a narrower question about the output. Given the telemetry a service has emitted, can a responder join each failure event to a request, and does each failure carry structured evidence of its cause? Distributed tracing makes the first property routine in principle [9]. In practice teams emit heterogeneous JSON logs, logfmt lines and OTLP exports, under field names that follow no single convention. The OpenTelemetry semantic conventions [10] standardise names for the teams that adopt them.

TELEMETRY-CONTRACTS treats each such property as a contract clause over a finite set of events and reports violations. The checks run on whatever a project ships, so they apply to repositories whose authors never used the tool. Our claim is that the two failure-path properties can be checked on raw, convention-free telemetry as accurately as a zero-shot LLM judge, at the cost and determinism of a linter. The contributions are:

- six gap classes with written definitions, and a checker that reads JSONL, JSON arrays, logfmt and OTLP exports without configuration (Sections III and IV);

- a gold-set comparison against two rule baselines and two zero-shot LLMs, with every prompt and raw response released;
- a study over 58 pinned public repositories with a validated stratified sample of 60 findings and a released annotation protocol.

We address three questions:

- **RQ1.** How accurate are the checks on a labelled set of telemetry samples?
- **RQ2.** How do they compare with rule baselines and with zero-shot LLMs given the same definitions?
- **RQ3.** When run on public repositories, how often do the findings match their definitions?

II. RELATED WORK

Logging practice and log enhancement. Empirical studies characterise how and where developers log in C/C++, Java, industrial and mobile code [1], [2], [6], [3], and what logging costs and buys [11]. LogEnhancer adds causally relevant variables to existing log statements [4]. Errlog inserts log statements at exception and error sites [5]. Learning-based approaches recommend where to log [7], [8], and LLM-based approaches generate the statements themselves [12], [13]. Chen and Jiang detect anti-patterns in logging code [14]. All of these analyse or change source code. **TELEMETRY-CONTRACTS** examines the emitted output instead, so it checks repositories in any language without building them.

Log analysis. Automated log analysis parses, mines and detects anomalies in logs once they exist [15], [16], and Loghub provides datasets for that work [17]. These techniques take the logs as given. **TELEMETRY-CONTRACTS** checks whether the logs can answer specific incident questions.

Specification mining and runtime verification. Specification miners infer temporal or invariant specifications from traces [18], [19], [20]. Runtime verification checks traces against temporal-logic specifications [21], [22], and hyper-properties capture information-flow requirements over sets of traces [23]. **TELEMETRY-CONTRACTS**’s contract inference and temporal clauses build on these ideas. The checks evaluated here are per-event properties.

Conventions and scanners. The OpenTelemetry semantic conventions [10] define field names, and our semconv-only baseline approximates a conformance linter. Secret scanners such as detect-secrets and gitleaks look for credentials in source files. The sensitive-value check applies similar patterns to telemetry values.

III. GAP CLASSES AND CHECKS

An *event* is a JSON object after format normalisation (Section IV). A *failure event* has severity error, critical or fatal, or a status/outcome field whose value denotes failure. The checks are:

- **missing-correlation:** A failure event with no correlation identifier (`trace_id`, `traceId`, `request_id`, `span_id` or a recognised alias).
- **missing-error-evidence:** A failure event with no error code, exception type or status field.
- **missing-duration:** A span event with no duration or latency measure.
- **sensitive-values:** A scalar value matching a raw-secret or PII pattern, namely an email, bearer token, JWT, password assignment, or a card or phone number in a field with a card or phone name.
- **unclassified-sensitive:** A field whose name denotes a tenant, customer, account or user identifier, or another sensitive attribute, emitted without redaction, hashing or a privacy classification.
- **unbounded-cardinality:** A metric label whose distinct-value count is high relative to the sample, reported only above a minimum sample size.

The first three checks are presence checks over field names, resolved through an alias table of common correlation, error and duration fields. The last three are pattern or statistical heuristics tuned for precision. Every finding records its source file, source line, check code and a remediation hint, and can be exported as SARIF for code-scanning pipelines.

IV. IMPLEMENTATION

TELEMETRY-CONTRACTS is a standard-library-only Python package with a command-line interface and a GitHub Action. Adapters read JSONL, JSON arrays, logfmt and OTLP/JSON collector exports. They map common aliases (`level/severity`, `msg/message`, `service.name`, `traceId`) and infer the signal kind with a confidence score. The `scan-repo` command clones a repository at a commit, discovers candidate telemetry files with a bounded walk that skips configuration, lock files and vendored directories, and checks each file. For each check code it keeps at most 25 example findings per file, so per-file counts are lower bounds. The `corpus-run` command scans a pinned-commit manifest with a content-addressed cache. Every aggregate uses integer or fixed-point arithmetic, so a rerun on the same inputs is byte-identical. The repository’s CI checks this against a SHA-256 manifest of 13 generated artifacts.

V. EVALUATION

All numbers below are regenerated by the commands in REPRODUCE.md. Raw outputs are under `results/` and `reports/`.

TABLE I

GOLD SET (N = 112). COVERED-CLASS F1 EXCLUDES CLASSES A METHOD CANNOT EXPRESS. DISCORDANT PAIRS: ITEMS ONLY THE TOOL GETS RIGHT / ITEMS ONLY THE OTHER METHOD GETS RIGHT.

Method	P	R	F1	Cov. F1	Disc. (p)
TELEMETRY-CONTRACTS	.962	.944	.953	.953	–
rule-light	.857	.667	.750	.809	20/1 (<.001)
semconv-only	.628	.500	.557	.771	38/0 (<.001)
Claude Haiku 4.5	.982	1.00	.991	.991	0/4 (.125)
GPT-4.1-mini	.980	.926	.952	.952	3/3 (1.00)

A. RQ1: Gold-set accuracy

Setup. The gold set (`benchmarks/ground_truth/curated.jsonl`) has 112 (sample, class) pairs, 54 positive and 58 negative, covering all six classes. Each label records whether the class is present according to the meaning of its definition, independent of the detector’s field lists. Five *hard* items probe the edges of field-keyed checking, such as a correlation id under the key `correlationGuid` and a base64-encoded credential. The tool’s author wrote the labels.

Results. The tool scores precision 0.962 (51/53, Wilson 95% CI 0.87–0.99), recall 0.944 (51/54, 0.85–0.98) and F1 0.953. Per-class F1 ranges from 0.923 (unbounded-cardinality) to 1.000 (missing-duration). The five errors are the five hard items. Two are false positives, an unrecognised correlation key and error evidence present only in free text. Three are false negatives, a base64 secret, an unrecognised tenant field and a cardinality sample below the trust threshold. Each error corresponds to a field name or encoding outside the alias and pattern tables.

B. RQ2: Rule baselines and LLMs

Setup. Every method receives only the events and the class name, never the label, and is scored by the same code. *rule-light* checks only for obvious field names. *semconv-only* flags missing OpenTelemetry convention fields, and its three classes without a convention field are reported out of scope. For the LLM rows we sent each item once through OpenRouter to `anthropic/claude-haiku-4.5` and `openai/gpt-4.1-mini`, at temperature 0 with at most 200 output tokens. The prompt states the class name, its definition and the events as JSON, and asks for YES or NO plus one sentence. All 224 responses parsed. The raw responses, served model ids, token counts and costs are in `results/llm_baseline/`. Both runs together cost US\$0.077. The comparison replays the recorded verdicts offline. Significance is an exact two-sided McNemar test on the paired per-item outcomes.

Results. Table I shows the results. The tool outperforms both rule baselines significantly. Its accuracy is statistically indistinguishable from both LLM judges. Claude Haiku 4.5 differs from the tool on four hard items, where a model reading the definition recognises keys outside the alias table. GPT-4.1-mini resolves three hard items and misses three unclassified-identifier positives that the tool catches. The fifth hard item,

TABLE II

RQ3: PREVALENCE OVER 56 TELEMETRY-BEARING REPOSITORIES, AND VALIDATED PRECISION ON 15 SAMPLED FINDINGS PER CLASS (WILSON 95% CI). FINDING COUNTS ARE CAPPED AT 25 PER CHECK PER FILE. BORDERLINE (SESSION-ID) CASES COUNT AS INCORRECT.

Class	Repos	Findings	Matches definition
missing-correlation	10	171	13/15 (.62-.96)
missing-error-ev.	10	127	12/15 (.55-.93)
sensitive-values	10	110	9/15 (.36-.80)
unclassified-sens.	19	1,861	1/15 (+9 borderline)
All sampled			35/60 (.46-.70)

error evidence present only in free text, is labelled negative and every method flags it. The tool delivers this accuracy with a linter’s operating profile. It runs offline with no API key, gives the same output on every run, and makes no model calls. An LLM judge needs one call per (event sample, class) pair, about 300 prompt tokens per item on this set, which on a repository scan grows with the number of events.

C. RQ3: Corpus study with validated findings

Setup. The corpus manifest pins 59 public repositories at full commit SHAs, of which 58 could be cloned (one was deleted upstream). They are hosted on GitHub (56), GitLab (1) and Bitbucket (1), and were selected by a curation script that searched for repositories shipping parseable telemetry files. Of these, 34 are primarily Python, and the rest include Go, Rust, TypeScript and Java. Running `corpus-run` over the 58 took 8.5 minutes on a laptop. We extracted all 2,269 findings with their source events and drew a stratified sample of 15 findings per class that fired, with at most two per (repository, class), ordered by a hash.

Validation procedure. An LLM agent inspected each sampled finding. It read the flagged event in the pinned checkout and the class definition, then assigned three labels. The first is whether the event matches the definition (yes, no or borderline). The second is the kind of file (runtime log written by the project, sample telemetry, test fixture, or non-telemetry). The third is whether a maintainer would treat the finding as a defect in telemetry the project emits. The labels and a one-line rationale for each are in `results/rq3/validation_labels.csv`. A blinded packet and agreement script for independent human raters are in `docs/evaluation/annotation/`.

Results. Table II summarises the scan and the validation. The scan found no missing-duration or unbounded-cardinality findings. Of the 14 telemetry-bearing repositories with at least one failure event, 10 have a failure event without a recognised correlation id.

The failure-path checks matched their definitions in 25 of 30 sampled findings (Wilson 95% CI 0.66–0.93). Four of the five mismatches come from field placement. Java logback events hold `trace_id` under a nested `mdc` object (one finding) or the exception class under a nested `throwable` object (two findings), and one OTLP record carries the convention attribute

`error.type`. Descending into nested objects and adding `error.type` to the alias table covers all four. The fifth is a disaster-recovery drill specification parsed as a failure event because of its category name.

The privacy checks matched their definitions in 9 of 15 sensitive-value findings and 1 of 15 unclassified-identifier findings, with nine more `session_id` fields judged borderline. Most mismatches come from datasets stored as JSON, where the password and bearer patterns match prose or code text, and from the `token` name heuristic matching token counts.

The file-kind labels separate project telemetry from demo data. Of the 60 sampled findings, 21 came from runtime logs the project wrote, 22 from demo telemetry, 5 from test fixtures and 12 from other JSON such as datasets. Seven were defects in the project’s own telemetry. Four are committed error-log entries with no exception type or error code, two are failures without request ids, and one is a committed agent prompt log containing the author’s personal email address.

Answer. On public repositories the failure-path checks match their definitions in 25 of 30 sampled findings, and four of the five mismatches are covered by two alias extensions.

VI. THREATS TO VALIDITY

Labels. The gold-set labels were written by the tool’s author, who also wrote the hard items, and the RQ3 labels come from a single LLM-assisted rater. The released packet and agreement script support an independent human rating of both. The LLM judges in RQ2 and the LLM rater in RQ3 may share failure modes, although the two tasks use different inputs and questions.

Sample sizes. With 15 findings per class, the intervals in Table II are 0.34 to 0.44 wide. The McNemar tests in RQ2 have little power at $n = 112$ with few discordant pairs.

LLM baseline. We used one zero-shot prompt per model with no tuning, and a single recorded sample per item. The committed log is the record of this run.

Corpus. The corpus was selected for parseable telemetry, not sampled at random, and is dominated by Python projects (34 of 58). Prevalence figures describe this corpus.

VII. CONCLUSION

TELEMETRY-CONTRACTS checks failure-path telemetry in data projects already emit, without configuration. On the gold set it matches two zero-shot LLM judges (F1 0.953 against 0.991 and 0.952, with no significant difference) while running offline and deterministically at no per-event cost. On 58 public repositories its correlation and error-evidence checks matched their definitions in 25 of 30 validated findings. The gold set, corpus manifest, raw model outputs and annotation packet are released for reuse.

ARTIFACT AVAILABILITY

Source, gold set, corpus manifest, LLM prompts and raw outputs, corpus findings and validation labels: <https://github.com/thehalleyoung/telemetry-contract-semantics>. See `REPRODUCE.md`.

REFERENCES

- [1] D. Yuan, S. Park, and Y. Zhou, “Characterizing logging practices in open-source software,” in *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, 2012, pp. 102–112.
- [2] B. Chen and Z. M. J. Jiang, “Characterizing logging practices in Java-based open source software projects – a replication study in Apache Software Foundation,” *Empirical Software Engineering*, vol. 22, no. 1, pp. 330–374, 2017.
- [3] Y. Zeng, J. Chen, W. Shang, and T.-H. P. Chen, “Studying the characteristics of logging practices in mobile apps: A case study on F-Droid,” *Empirical Software Engineering*, vol. 24, no. 6, pp. 3394–3434, 2019.
- [4] D. Yuan, J. Zheng, S. Park, Y. Zhou, and S. Savage, “Improving software diagnosability via log enhancement,” in *Proceedings of the 16th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2011, pp. 3–14.
- [5] D. Yuan, S. Park, P. Huang, Y. Liu, M. M. Lee, X. Tang, Y. Zhou, and S. Savage, “Be conservative: Enhancing failure diagnosis with proactive logging,” in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2012, pp. 293–306.
- [6] Q. Fu, J. Zhu, W. Hu, J.-G. Lou, R. Ding, Q. Lin, D. Zhang, and T. Xie, “Where do developers log? An empirical study on logging practices in industry,” in *Companion Proceedings of the 36th International Conference on Software Engineering (ICSE Companion)*, 2014, pp. 24–33.
- [7] J. Zhu, P. He, Q. Fu, H. Zhang, M. R. Lyu, and D. Zhang, “Learning to log: Helping developers make informed logging decisions,” in *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2015, pp. 415–425.
- [8] Z. Li, T.-H. P. Chen, and W. Shang, “Where shall we log? Studying and suggesting logging locations in code blocks,” in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020, pp. 361–372.
- [9] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, “Dapper, a large-scale distributed systems tracing infrastructure,” Google, Inc., Tech. Rep. dapper-2010-1, 2010.
- [10] OpenTelemetry Authors, “OpenTelemetry semantic conventions,” <https://opentelemetry.io/docs/specs/semconv/>, accessed 2026-10-06.
- [11] H. Li, W. Shang, B. Adams, M. Sayagh, and A. E. Hassan, “A qualitative study of the benefits and costs of logging from developers’ perspectives,” *IEEE Transactions on Software Engineering*, vol. 47, no. 12, pp. 2858–2873, 2021.
- [12] J. Xu, Z. Cui, Y. Zhao, X. Zhang, S. He, P. He, L. Li, Y. Kang, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, “UniLog: Automatic logging via LLM and in-context learning,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024, pp. 14:1–14:12.
- [13] Y. Li, Y. Huo, Z. Jiang, R. Zhong, P. He, Y. Su, L. Briand, and M. R. Lyu, “Exploring the effectiveness of LLMs in automated logging generation: An empirical study,” 2023.
- [14] B. Chen and Z. M. J. Jiang, “Characterizing and detecting anti-patterns in the logging code,” in *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2017, pp. 71–81.
- [15] S. He, P. He, Z. Chen, T. Yang, Y. Su, and M. R. Lyu, “A survey on automated log analysis for reliability engineering,” *ACM Computing Surveys*, vol. 54, no. 6, pp. 130:1–130:37, 2022.
- [16] J. Zhu, S. He, J. Liu, P. He, Q. Xie, Z. Zheng, and M. R. Lyu, “Tools and benchmarks for automated log parsing,” in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 121–130.
- [17] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, “Loghub: A large collection of system log datasets for AI-driven log analytics,” in *Proceedings of the 34th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, 2023, pp. 355–366.
- [18] G. Ammons, R. Bodík, and J. R. Larus, “Mining specifications,” in *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2002, pp. 4–16.
- [19] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, “The Daikon system for dynamic detection of likely invariants,” *Science of Computer Programming*, vol. 69, no. 1–3, pp. 35–45, 2007.
- [20] C. L. Goues and W. Weimer, “Specification mining with few false positives,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. LNCS, vol. 5505, 2009, pp. 292–306.
- [21] M. Leucker and C. Schallhart, “A brief account of runtime verification,” *The Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.
- [22] A. Bauer, M. Leucker, and C. Schallhart, “Runtime verification for LTL and TLTL,” *ACM Transactions on Software Engineering and Methodology*, vol. 20, no. 4, pp. 14:1–14:64, 2011.
- [23] M. R. Clarkson and F. B. Schneider, “Hyperproperties,” *Journal of Computer Security*, vol. 18, no. 6, pp. 1157–1210, 2010.